Group 17: File Sharing Program

Project 4

Computer Science 313

Members (Current and Historical Roles)

	Project 1	Project 2	Project 3	Project 4
Ryan Allen Student Email: 26221527@sun.ac.za	**Role: Server** Server.java Experiment 1 Experiment 2	**Role: Server** RBUDPReceiver.java Experiment 1	**Role: Client** Client.java Experiment 2	**Role: Client** P2PTransferEngine.java Experiment 2
Courtney Campbell Student Email: 27906124@sun.ac.za	**Role: Client** Client.java LoginController.java ChatApp.java Login.fxml Experiment 1 Experiment 2	**Role: Client** Packet.java Sender.java Experiment 3	**Role: Client** AudioConfig.java AudioSender.java AudioReciever.java AudioMixer.java	**Role: Security** EncryptedMessage.java EncryptionManager.java KeyHandler.java Client.java Experiment 2
Erin Painting Student Email: 27023931@sun.ac.za	**Role: Client** ChatApp.java client.java ChatController.java ChatUI.fxml Message.java README.md	**Role: UI** RecieverController.java SenderController.java MainApp.java ReceiverUI.java SenderUI.java README.md	**Role: UI** ChatController.java ChatUI.fxml WindowManager.java ServerApp.java ServerController.java ServerUI.fxml Experiment 3	**Role: UI** Experiment 1 App.java LoginController.java LoginView.fxml MainController.java MainView.fxml
Kayla De Beer Student Email: 27480216@sun.ac.za	**Role: Server** Server.java Experiment 3	**Role: Server** RBUDPReceiver.java Experiment 2 Experiment 4	**Role: Server** Server.java Experiment 1	**Role Server** Server.java Experiment 3

1. Overview

The primary objective of this project is the design and implementation of a decentralized peer-to-peer (P2P) file-sharing system. This program allows anonymous users to connect to a central server to search for files and facilitate direct transfers between clients. While a central server orchestrates the network by managing connections and handling search requests, it remains a minimal component that does not store files or maintain a global list of online users to ensure peer anonymity.

A critical focus of this implementation is the integration of cybersecurity practices. Communication between peers and the server is secured using encryption protocols to protect message keys and data integrity. Furthermore, the system is designed to operate over a virtual network using TailScale, ensuring secure connectivity regardless of physical location. This project serves as a practical exploration of network synchronization, concurrent file streaming (supporting simultaneous uploads and downloads) and the implementation of secure "introduction" protocols between anonymous nodes.

2. AI Declaration

- Gemini was used to help with skeleton code for EncryptedMessage.java, EncryptionManager.java and KeyHandler.java.
- Gemini was used to help with understanding encryption and security in general and for this implementation.
- Gemini was used to clean comments
- Claude was used to help debug when compiling errors were received
- ChatGPT was used to explain and break down the project and what needed to be implemented
- Claude was used to help create the Makefile
- ChatGPT was used to correct grammar and make the report consistent
- ChatGPT was used to help write a clean README
- Claude was used to help with styling consistency in the UI
- Gemini was used to teach the concepts as well as skeleton for the server side
- ChatGPT was used to confirm correct interpretation of experiment results
- Gemini was used to assist in making the experiment files
- Claude helped with giving a basic UI design and colors
- ChatGPT gave the ideas for the experiments
- Gemini helped design the search logic on the server side (gave helper classes)
- Gemini was used to understand the flow of the project with regards to security
- Gemini was used to advise on where to add timer for experiment 3
- Gemini was used to interpret experiment results
- ChatGPT was used to write report conclusion

3. Experiments

Experiment 1: search results time and number of files in a folder

Hypothesis

Search response time will remain roughly constant regardless of the number of shared files because the server waits for a fixed 1-second collection window before returning results. Therefore, increasing the number of files in the shared folder should not significantly affect the overall search response time.

Method

1. Make a folder that contains a number of files. Using the following command:
   ```
   mkdir -p test_files
   for i in $(seq 1 500); do touch test_files/file_$i.txt; done
   ```
2. Start the server and two clients.
3. Client A shares the folder.
4. Client B sends a search and times how long until results arrive.
5. Repeat 4 times per file count, record the average.

Variables:

Independent Variable: Number of files in the shared folder (10, 50, 100, 200, 500)

Dependent Variable: Time from sending the search request to receiving results back (milliseconds)

Controlled Variables:

- Same machine, same server
- Same search query each time
- Always searching for the same filename
- Encryption always on

Visualisation

Number of files	Trial 1 (ms)	Trial 2 (ms)	Trial 3 (ms)	Trial 4 (ms)	Average (ms)
10	**1068**	**1048**	**1005**	**1004**	**1031**
50	**1004**	**1003**	**1003**	**1003**	**1003**
100	**1003**	**1004**	**1003**	**1002**	**1003**
200	**1003**	**1004**	**1006**	**1004**	**1004**
500	**1004**	**1003**	**1004**	**1003**	**1004**

Conclusion

The results supported the hypothesis. Response time remained stable across all file counts, averaging approximately 1000–1030 ms regardless of the number of shared files. This confirms that the server's fixed 1-second collection window dominates the total response time. Increasing the number of shared files from 10 to 500 did not significantly affect search performance, showing that the system scales well for this range of file counts.

Experiment 2: File Size vs Transfer Time

Hypothesis: Transfer time will increase linearly with file size. Because the system encrypts and transmits data in fixed 4KB chunks, the processing overhead scales directly with the payload. Therefore, a 100MB file should take approximately 10 times longer to transfer than a 10MB file.

Variables:

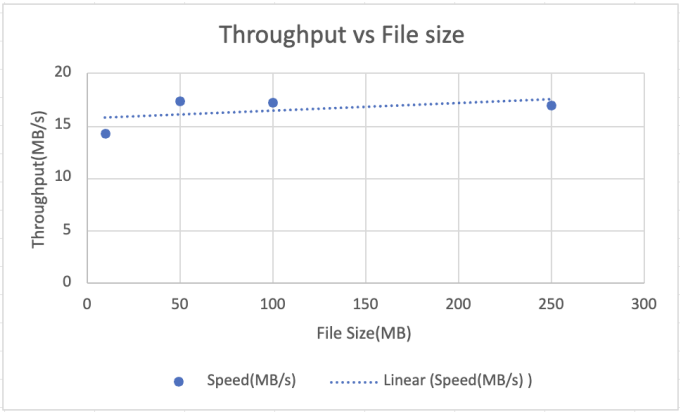
- Independent: File size (10MB, 50MB, 100MB, 250MB).
- Dependent: Total transfer time (milliseconds), Speed (MB/s).

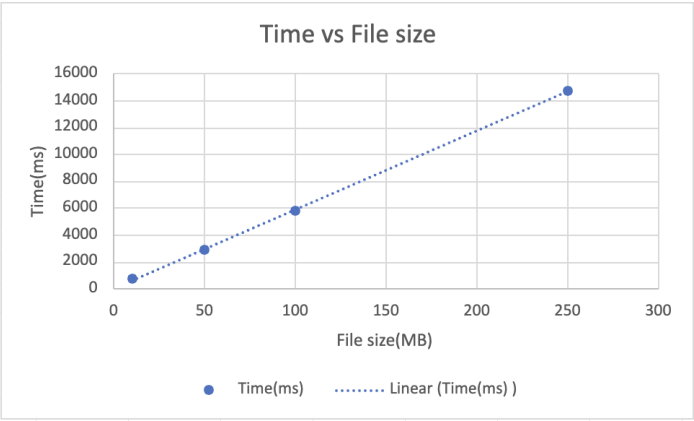
Method:

1. Generate four dummy files via the terminal (10MB, 50MB, 100MB, and 250MB).
2. Host the test folder on Client A (Uploader).
3. Download each file sequentially on Client B (Downloader).
4. Record the exact transfer time outputted by the internal System.currentTimeMillis() timer in the terminal.

Visualization:

File size (MB)	10	50	100	250
Time(ms)	701	2896	5821	14732
Speed (MB/s)	14.265	17.265	17.179	16.970





Conclusion:

The data strongly supports the hypothesis that transfer time increases linearly with file size. As the payload scaled, the processing and transmission times scaled proportionally. Furthermore, the throughput analysis reveals a highly stable system: after an initial connection and handshake overhead, which caused a slight dip in the 10MB speed, the application quickly reaches and sustains a consistent transfer rate of approximately 17 MB/s. This confirms that the AES encryption and chunking mechanics introduce a predictable, linear overhead without bottlenecking or degrading over longer, continuous transfers.

Experiment 3: Impact of AES Encryption on File Transfer Speed

This experiment tests the "cost" of the security gate.

- **Question**: How much overhead does full AES payload encryption add to the total file transfer time?
- **Hypothesis**: Encrypted file transfers will take approximately 10-15% longer than unencrypted transfers due to the CPU cycles required for the EncryptionManager to process data chunks.
- **Independent Variable**: Encryption status (Unencrypted vs. Encrypted).
- **Dependent Variable**: Total time (milliseconds) to transfer a 50MB file.
- **Method**:
    - Transfer a 50MB file between two clients with encryption
    - Measure the total transfer time as well as the time taken to AES encryption
    - Approximate the unencrypted time as: total transfer time – AES encryption time
    - Measure the time for 3 trials
    - Calculate overhead as a percentage
- **Visualisation**:
  Below is the table detailing the time measurements used for the approximation

	Trial number		
	Trial 1	Trial 2	Trial 3
Total Transfer Time (ms)	1534	1031	869
Cumulative AES Time (ms)	625.22	321.72	230.28

Below is the table with experiment results

	Transfer Time (ms)		
	Unencrypted	Encrypted	Percentage Difference (Overhead)
Trial 1	908.78	1534	68.80%
Trial 2	709.28	1031	45.36%
Trial 3	638.72	869	36.05%

- **Conclusion:** The percentage overhead is noticeably greater than the prediction – average 50% compared to 10-15%. Although the percentage overhead decreases with repeated trials. Note that these are not independent trials because of the slow start mechanism. An argument can be made that the overhead could approach an amount close to that which was hypothesised through increased trials and exclusion of the first few trials as outliers.

Conclusion

This project successfully demonstrated the design and implementation of a secure peer-to-peer file sharing system. The experiments showed that the architecture scales effectively for file searching, maintains stable transfer speeds across increasing file sizes, and supports reliable concurrent communication between peers. While AES encryption introduced a noticeable performance overhead, the results confirmed that the added security cost remained predictable and manageable. Overall, the project achieved its goals of combining networking, synchronization, and cybersecurity concepts into a functional and secure distributed application operating over a virtual network environment.